

Solitito — podsumowanie projektu

System rozpoznawania akordów gitarowych w czasie rzeczywistym

Wersja 0.2.0, sierpień 2026

1. Charakterystyka systemu

Solitito jest trenażerem gitarowym działającym w czasie rzeczywistym, zaimplementowanym w języku Rust. Program pobiera sygnał z mikrofonu lub interfejsu audio, rozpoznaje wykonywany materiał i prowadzi użytkownika przez standardy jazzowe, interwały, skale oraz arpeggia.

Rozpoznawanie realizuje sieć neuronowa o 7,3 mln parametrów, wyeksportowana do formatu ONNX. Całość przetwarzania — DSP, inferencja oraz interfejs użytkownika — wykonywana jest lokalnie na procesorze, bez połączenia sieciowego i bez usług zewnętrznych.

System udostępnia cztery tryby pracy:

- **Akordy** — pełne standardy jazzowe. Kolor zielony oznacza trafienie dokładne, żółty — triadę lub typowe zastępstwo, czerwony — akord rozpoznany przy sygnale zbyt słabym, by go zatwierdzić.
- **Interwały** — składniki akordu wykonywane pojedynczo, z możliwością wyboru ćwiczonych stopni.
- **Skale** — sekwencyjne przechodzenie dźwięków zgodnie z definicją gamy.
- **Arpeggia** — składniki akordu w sekwencji, na zadanej progresji.

Prace nad projektem rozpoczęto w grudniu 2025 roku. Niniejszy dokument przedstawia architekturę systemu, przebieg prac oraz decyzje projektowe wraz z ich uzasadnieniem.

2. Metodyka prac

W projekcie przyjęto zasadę, że każda zmiana wymaga uzasadnienia pomiarowego, a nie

hipotezy. W praktyce oznaczało to opracowanie zestawu **sond** — skryptów odpowiadających na pojedyncze pytanie niewielkim kosztem obliczeniowym, bez konieczności ponownego treningu.

sonda	zagadnienie
<code>verify_annotations.py</code>	czy etykiety opisują zawartość audio?
<code>probe_root.py</code>	jak często pryma wskazana etykietą faktycznie brzmi w oknie?
<code>probe_sources.py</code>	która adnotacja akordowa zbioru GuitarSet jest użyteczna?
<code>probe_quality.py</code>	z jakiego źródła wyprowadzać jakość akordu?
<code>inspect_jams.py</code>	jaka jest rzeczywista zawartość plików JAMS?

Metodyka ta wykazała skuteczność wielokrotnie. Odnotować należy również jej rewers: **hipotezy formułowane przed wykonaniem pomiaru okazywały się błędne w sposób systematyczny**. Zestawienie tych przypadków zawiera rozdział 9.

3. Zbiór syntetyczny

Skrypt `dataset_generator_v2.py` wytwarza w jednym przebiegu plik Guitar Pro **oraz** komplet adnotacji.

3.1. Zawartość

394 bloki o czasie trwania 6 sekund (3 takty przy 120 BPM), obejmujące:

- 12 prym $\times$ {maj, min, maj7, dom7, min7, m7b5, dim7, sus4, aug} w kilku pozycjach na gryfie,
- wszystkie 96 pojedynczych dźwięków (6 strun $\times$ 16 progów).

Struktura bloku: takt pierwszy stanowi atak, drugi — wybrzmienie (tie), trzeci — ciszę. Adnotacja obejmuje przedział [`start + 0,05 s`, `+3,2 s`], to jest atak wraz z sustainem, z pominięciem ogona zaniku.

3.2. Weryfikacja chwytów

Przed rozpoczęciem generacji skrypt sprawdza **każdy z 21 ruchomych chwytów na każdym progu**, weryfikując, czy chwyt realnie daje deklarowane interwały. Błąd w tabeli chwytów zatrzymuje generację zamiast propagować się do zbioru danych.

3.3. Render

Ścieżka gitary jest eksportowana jako sygnał DI i renderowana w środowisku DAW przy użyciu **NAM** w dwóch wariantach:

- `synth_dataset_clean.wav` — Fender Deluxe Reverb, brzmienie czyste,
- `synth_dataset_eob.wav` — na granicy przesteru.

Zalecaną częstotliwością próbkowania jest **48 kHz**, z dwóch niezależnych powodów. NAM pracuje natywnie w 48 kHz, wobec czego przy 44,1 kHz wtyczka wykonuje resampling wewnętrzny. Ponadto decymacja do 16 kHz, na których pracuje model, jest przy 48 kHz dokładna ($48000/16000 = 3$), a przy 44,1 kHz — nie ($2,75625$).

3.4. Kalibracja i weryfikacja

Tryb `--calibrate <wav>` wyznacza moment pierwszego ataku na wypadek, gdyby środowisko DAW dodało ciszę na początku pliku. Obsługiwane są formaty PCM 16/24/32-bit oraz zmiennoprzecinkowe 32/64-bit, mono i stereo — przy użyciu własnego czytnika WAV, bez zależności zewnętrznych.

Skrypt `verify_annotations.py` porównuje etykietę z **faktyczną zawartością audio**: dominującą klasą wysokości w oknie zestawia się z prymą wskazaną etykietą. Jedyną zależnością jest numpy.

Wartości odniesienia uzyskane z renderu v2:

	clean	eob	wartość losowa
--	--------------	------------	-----------------------

top1	87%	77%	~8%
-------------	-----	-----	-----

top3	100%	98%	~25%
-------------	------	-----	------

Miarodajna jest wartość top3, ponieważ w akordzie pryma bywa cichsza od tercji lub kwinty. Wartość `top3 > 75%` kwalifikuje zbiór do treningu; `top3 ≈ 25%` oznacza, że etykiety nie opisują sygnału.

Skrypt weryfikuje ponadto okres bloków wyznaczony z obwiedni energii oraz przesunięcie pomiędzy początkiem adnotacji a atakiem. Oba testy są **komplementarne**: przesunięcie adnotacji o całkowitą wielokrotność bloku trafia w atak sąsiedniego bloku i pozostaje niewykrywalne dla testu czasowego. Wykrywa je dopiero porównanie etykiet z zawartością sygnału.

Przyjęto zasadę nadrzędną: **generator wypisuje etykiety wprost, a niezależny skrypt weryfikuje ich zgodność z audio**. Żaden krok przetwarzania nie odtwarza etykiet z sygnału.

4. Zbiór GuitarSet

GuitarSet obejmuje 360 nagrań z adnotacjami w formacie JAMS, zarejestrowanych przetwornikiem heksafonicznym. Jest to jedyne źródło materiału z rzeczywistego instrumentu wykorzystane w projekcie. Doprowadzenie go do postaci użytecznej wymagało czterech przebiegów treningowych.

Poniżej opisano cztery właściwości zbioru, których pominięcie obniża dokładność modelu.

4.1. Połowa zbioru nie zawiera akordów

Każdy fragment zarejestrowano dwukrotnie: jako `_comp` (akompaniament) oraz `_solo` (improvizacja jednogłosowa). **Adnotacja akordowa jest w obu przypadkach identyczna** — opisuje progresję, nad którą wykonawca improwizował.

Trening głowic akordowych na plikach solo sprowadza się do uczenia modelu, że pojedynczy dźwięk stanowi pełny akord jazzowy. Materiał solowy obejmuje 180 z 360 plików.

Odfiltrowanie tych plików przesunęło wskaźnik `Exact` z **44,8% na 82,3%** w jednym przebiegu.

Cele **pitch** pochodzące z plików solo pozostają w pełni poprawne — jest to rzeczywiste wykonanie jednogłosowe z dokładnymi adnotacjami nutowymi, a więc materiał odpowiedni dla detektora pojedynczych dźwięków. Trener zachowuje zatem stratę pitch na oknach solowych i maskuje wyłącznie prymę oraz jakość (`GUITARSET_SOLO_MODE = "mask_chord"`).

Odnosić należy konsekwencję pośrednią. Sonda `probe_root.py` wyznaczała początkowo słyszalność prymy na wszystkich 360 plikach, uzyskując sufit 64,1%. Na tej podstawie sformułowano wniosek, że wykonawcy jazzowi stosują voicingi bez prymy, a nazwa akordu

nie jest funkcją sygnału. Wniosek ten okazał się nieprawidłowy: po odfiltrowaniu materiału solowego pryma brzmi w 97% okien akompaniamentu.

4.2. Adnotacje akordowe występują w dwóch wariantach

Każdy plik zawiera adnotację `instructed` (akord wynikający z zapisu) oraz `performed` (transkrypcja wykonania). Liczba segmentów jest identyczna, rozkład etykiet — różny:

jakość	<code>instructed</code>	<code>performed</code>	różnica
maj	2640	2106	-534
min	960	460	-500
min7	0	360	+360
maj7	0	430	+430
dom7	480	694	+214
m7b5	240	134	-106
sus	0	132	+132
razem	4320	4320	0

Suma segmentów pozostaje niezmienną, wobec czego nie jest to wybór pomiędzy większą a mniejszą liczbą danych, lecz **przeetykietowanie tych samych nagrań**.

Wiersze `min` oraz `min7` należy rozpatrywać łącznie: **pięćset segmentów oznaczonych w zapisie jako `m` zostało wykonanych jako `m7`**. Trening na adnotacji `instructed` uczy model, aby voicing zawierający septymę małą klasyfikować jako zwykły akord molowy. Błąd ten był następnie obserwowany w aplikacji, gdzie akord `Gm7` rozpoznawano jako `Gm`.

Ponadto adnotacja `instructed` nie zawiera **ani jednego** wystąpienia klas `maj7` i `min7`. Do momentu przełączenia obie klasy pochodziły wyłącznie z dwóch renderów zbioru syntetycznego, to jest z jednego instrumentu przetworzonego przez jeden wzmacniacz. Skutkowało to wartością 100% na zbiorze walidacyjnym (ten sam instrument po obu stronach podziału) przy jednoczesnym braku odporności na materiał rzeczywisty.

Przełączenie na adnotację `performed` przesunęło wskaźnik `Exact` z **82,3% na 92,4%**. Dokładność rozpoznawania prymy pozostała bez zmian: obie adnotacje różnią się co do

prymy w 0 z 43 056 porównań.

4.3. Podział zbioru musi przebiegać po źródle

Losowe potasowanie listy segmentów akordowych i podział w proporcji 94/6 umieszcza sąsiadujące takty **tego samego nagrania** po obu stronach podziału — przy identycznym instrumencie, pomieszczeniu, mikrofonie i ujęciu, często przy tym samym akordzie występującym takt później.

W zbiorze syntetycznym zależność jest silniejsza: rendery `clean` i `eob` jednego bloku stanowią to samo wykonanie przetworzone przez inny wzmacniacz, a trafiły do zbioru treningowego i walidacyjnego niezależnie.

Zastosowane rozwiązanie polega na grupowaniu po źródle: całym pliku dla zbioru GuitarSet oraz całym bloku (obu renderach) dla zbioru syntetycznego.

Wszystkie wskaźniki walidacyjne ulegają po tej zmianie obniżeniu. Nie stanowi to regresji modelu, lecz usunięcie zawyżenia, które unieważniało wcześniejsze wnioski dotyczące generalizacji. Wartość `root_acc = 98%` uzyskana w przebiegu `take1` była w znacznej mierze artefaktem: przy adnotacjach `both` ten sam segment występował dwukrotnie, wobec czego identyczne okno trafiało zarówno do zbioru treningowego, jak i walidacyjnego.

4.4. Cele pitch wyznaczone z `note_midi`

Adnotacja akordowa opisuje **zamierzoną harmonię** w skali wielu sekund. Okno treningowe obejmuje 0,77 s i często nie zawiera etykietowanej septymy. Model był zatem karany za nieprzewidzenie dźwięku nieobecnego w sygnale — `recall septym` na zbiorze GuitarSet wynosił 32%.

Przetwornik heksafoniczny dostarcza adnotacji `note_midi`, opisujących rzeczywiste wykonanie na każdej strunie osobno. Wyznaczenie celów pitch na ich podstawie podniosło `recall septym` z 32% do 96%.

Przyjęty próg: dźwięk musi brzmieć przez co najmniej 25% okna (`NOTE_MIN_COVER`), aby zostać uwzględniony w celu.

5. Architektura

5.1. Ścieżka sygnału

```
wejście audio → resampling do 16 kHz → FFT (8192) → rzadkie pseudo-CQT → cechy →  
model ONNX
```

Resampling do 16 kHz. Transformata CQT obejmuje 6 oktafów począwszy od C1, wobec czego najwyższy bin przypada w okolicy 2 kHz, a więc znacznie poniżej granicy Nyquista wynoszącej 8 kHz. Pasma nie stanowi ograniczenia.

Pseudo-CQT. Zamiast właściwej transformaty o stałym współczynniku Q aplikacja mnoży widmo FFT przez wyznaczone uprzednio jądro: 144 biny, 24 na oktawę, co odpowiada rozdzielczości ćwierćtonowej. Jądro pochodzi z funkcji `librosa.filters.constant_q`, dzięki czemu aplikacja i trener wytwarzają identyczne cechy.

Cechy. 168 wartości na ramkę:

zakres	zawartość
--------	-----------

0–143	biny CQT po log-normalizacji
-------	------------------------------

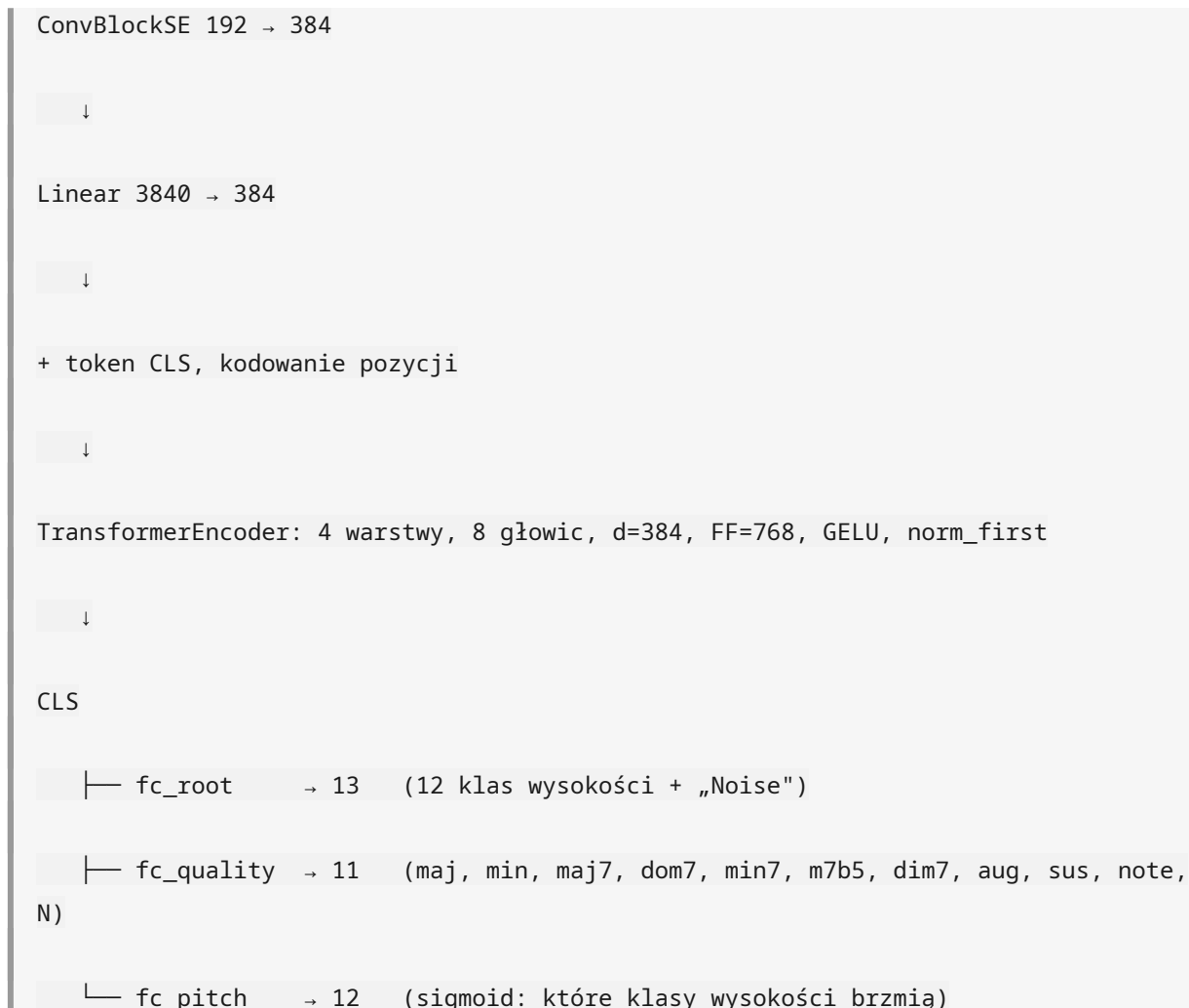
144–155	chroma (macierz <code>cq_to_chroma</code> , normalizacja maksimum w ramce)
---------	--

156–167	energia basowa (średnia z par binów 0–23)
---------	---

Model obejmuje 48 ramek historii przy skoku 256 próbek, co odpowiada **0,77 s**.

5.2. Sieć

```
wejście [48, 168]  
  
↓  
  
InstanceNorm2d  
  
↓  
  
ConvBlockSE 1 → 48 (Squeeze-and-Excitation)  
  
ConvBlockSE 48 → 96  
  
ConvBlockSE 96 → 192
```



Łączna liczba parametrów: 7 286 038.

5.3. Podział zadań pomiędzy głowicami

Rozróżnienie ról poszczególnych głowic ma charakter kluczowy i zostało potwierdzone pomiarem.

głowica	wynik	rola
pitch_logits	F1 0,909	które dźwięki brzmią — podstawa trybów Interwały, Skale i Arpeggia
root_logits	98,1%	nazwa prymy
quality_logits	~93%	rodzina akordu

Wczesna wersja aplikacji wyprowadzała jakość akordu z wektora pitch przy użyciu progów ustalonych ręcznie. Sonda `probe_quality.py` zestawiała trzy metody na tym samym checkpointcie:

metoda	dokładność
głowica <code>quality_logits</code>	80,5%
dopasowanie szablonów do przewidzianego pitch	66,0%
dopasowanie szablonów do rzeczywistego wektora pitch	59,2%

Głowica przewyższa dopasowanie szablonów do *dokładnie znanego* zbioru dźwięków o 21 punktów procentowych. Wyprowadza zatem z sygnału informację nieobecną w samym zbiorze klas wysokości: barwę, rozłożenie voicingu w rejestrze oraz kształt ataku.

Wniosek projektowy: głowica jakości pozostaje elementem koniecznym.

6. Trening

6.1. Fazy

Procedura treningowa obejmuje trzy fazy, przy czym zasadnicze znaczenie ma faza pierwsza.

faza	zakres	status
1	trening zasadniczy, 120 epok, cosine LR z rozgrzewką	jedyna wnosząca poprawę
2	strojenie progu głowicy pitch	skorygowana — sortowała po metryce niezależnej od progu
3	dostrajanie głowic przy zamrożonym enkoderze	wyłączona

Faza 2 skanowała progi w zakresie 0,30–0,70, optymalizując wskaźnik `exact`. Wskaźnik ten stanowi koniunkcję `argmax(root)` oraz `argmax(quality)`, wobec czego przyjmował identyczną wartość dla wszystkich 41 progów, a wybór był losowy. Obecnie sortowanie odbywa się po F1 głowicy pitch, na którą próg faktycznie oddziałuje.

Fazę 3 wyłączono po zmierzeniu jej efektów w trzech kolejnych przebiegach:

```
take2, 40 epok: pitch_f1 0,9318 → 0,9326 (+0,0008), exact 0,5455 → 0,5445
```

take3, 4 epoki: F1 0,933 → 0,931, exact 54,6% bez zmian

Enkoder pozostaje zamrożony, uczeniu podlegają wyłącznie głowice przy współczynniku uczenia $1e-5$. Faza nie dysponuje mechanizmem poprawy modelu, a jej koszt wynosi około 1,5 godziny obliczeń.

6.2. Funkcje straty i maskowanie

- **root** — CrossEntropy z wygładzaniem etykiet 0,05,
- **quality** — CrossEntropy z wygładzaniem, sampler ważony po klasie,
- **pitch** — Focal BCE ($\gamma = 2,0$, `pos_weight` 2,5), waga pomocnicza 0,7.

Zastosowano dwa mechanizmy maskowania, oba uzasadnione pomiarem.

MASK_ROOT_WHEN_SILENT — strata prymy wyznaczana wyłącznie na oknach, w których pryma faktycznie brzmi. Trening prymy na oknach jej pozbawionych nie prowadzi do wyuczenia percepcji, lecz do zapamiętania progresji zbioru GuitarSet, przy czym wspólny enkoder otrzymuje gradient sprzeczny z celem pitch.

GUITARSET_SOLO_MODE = "mask_chord" — pryma i jakość nie otrzymują gradientu z nagrań solowych; głowica pitch otrzymuje go bez zmian.

6.3. Augmentacja

- **przesunięcie wysokości** o $\pm N$ półtonów. Istotny szczegół implementacyjny: CQT oraz energia basowa przesuwane są **z wypełnieniem zerami**, chroma — **cyklicznie**. Chroma jest z definicji okrężna, CQT nie jest; zawinięcie pasma basowego na górę zakresu wprowadzałoby dźwięki nieobecne w sygnale.
- **maskowanie czasu i częstotliwości** (SpecAugment),
- **nachylenie widma oraz szum** — symulacja zróżnicowanych torów sygnału.

6.4. Bramka energetyczna

Parametr `ENERGY_KEEP_FRAC = 0,55` odrzuca okna o energii poniżej 55% wartości szczytowej segmentu. Uzasadnienie: w fazie zaniku septyma, będąca najcichszym składnikiem voicingu, zanika jako pierwsza, podczas gdy etykieta pozostaje niezmieniona. Brak bramki prowadziłby do systematycznego uczenia kolapsu `m7` → `m`.

6.5. Metryki

Metryki akordowe wyznaczane są **wyłącznie na oknach, w których etykieta opisuje sygnał**, z pominięciem okien solowych. Raportowane są dodatkowo w rozbiu na okna ze słyszalną prymą i bez niej, ponieważ wartość łączna miesza dwie odmienne populacje.

Wybór najlepszego checkpointu odbywa się według wskaźnika `composite = (root_audible + qual + exact) / 3`. Zastosowanie łącznego `root_acc` premiowałoby model skutecznie odtwarzający progresje zamiast modelu poprawnie analizującego sygnał.

Kontrola diagnostyczna `TRAIN`, wykonywana co 5 epok, wyznacza metryki na danych treningowych bez augmentacji. Odpowiada na pytanie, czy model jest w stanie odwzorować własne dane treningowe. Odpowiedź negatywna wskazuje na cechy lub etykiety jako źródło ograniczenia, nie na generalizację, i oznacza, że zwiększanie liczby epok jest bezcelowe.

7. Wyniki

Model `v2_take6`, walidacja z podziałem po źródle, z pominięciem okien solowych:

metryka	wartość
dokładność prymy	98,1%
pitch F1	0,909
trafienie dokładne (pryma i jakość)	92,4%

Dokładność w podziale na jakości przy najlepszym checkpointie: `dom7` 97%, `min7` 93%, `min` 92%, `sus` 91%, `maj` 89%, `maj7` 89%; klasy `m7b5`, `dim7` oraz `aug` powyżej 97%.

7.1. Przebieg prac

przebieg	zmiana	Exact
take1–take3	różne, przy podziale z przeciekiem	nieporównywalne
take4	podział po źródle — punkt odniesienia	44,8%
take5	maskowanie nagrań solowych	82,3%

take6

adnotacje performed

92,4%

7.2. Ograniczenie dokładności

Różnica pomiędzy zbiorem treningowym a walidacyjnym w zakresie jakości wynosi **6,5 punktu procentowego** (99,2% wobec 92,7%). Model odwzorowuje dane treningowe. Odpowiada to profilowi ograniczenia przez **generalizację**, nie przez pojemność architektury.

Wniosek praktyczny: zwiększenie liczby epok ani rozmiaru modelu nie przyniesie poprawy. Poprawę przyniesie zwiększenie ilości zróżnicowanego materiału z rzeczywistego instrumentu.

8. Aplikacja

Dokładność rozpoznawania nie jest równoznaczna z użytecznością trenażera. Trzy zagadnienia okazały się mieć wagę porównywalną ze zmianami w modelu.

8.1. Wymóg pełnego okna kontekstowego

Trener wyznaczał okna **wyłącznie wewnątrz** wybrzmiewającego akordu (`range(start, koniec - 48)`). Po uderzeniu w struny bufor aplikacji przez 0,77 s zawiera częściowo ciszę; uwzględniając okno FFT (8192 próbki, czyli 512 ms), najstarsza ramka opisuje sygnał sprzed nawet 1,3 s. Stanowi to wejście spoza rozkładu treningowego.

Zaobserwowany objaw: akordy septymowe rozpoznawane były dopiero w fazie wybrzmiewania, to jest w pierwszym momencie, w którym okno zostaje w całości wypełnione akordem.

Zastosowane rozwiązanie: aplikacja nie kieruje zapytania do modelu, dopóki okno nie jest wypełnione sygnałem w 90%.

8.2. Zróżnicowany czas wybrzmiewania składników akordu

Podgląd diagnostyczny przytrzymanego akordu `Gm7`:

G m7 | min7=96% | b7=96 ← bezpośrednio po uderzeniu

G m7 | min7=82% | b7=76

G m7 | min7=52% | b7=52

G m | min=49% | b7=45 ← septyma wyciszona, model zmienia klasyfikację

Klasyfikacja modelu jest poprawna — w bieżącym oknie septyma faktycznie nie występuje. Akord nie zmienia jednak tożsamości w trakcie wybrzmiewania.

Zastosowane rozwiązanie: zatrask jakości. Mechanizm **załącza się** przy pewności $\geq 0,60$, natomiast **utrzymuje stan niezależnie od niej**. W fazie zaniku model raportuje uboższą jakość z pewnością rzędu 94–96%, wobec czego sam próg pewności nie stanowiłby zabezpieczenia. Zwolnienie zatrasku następuje przy nowym ataku lub przy zmianie prymy.

Zatrask załącza się dopiero po 48 klatkach od ataku. Wcześniej okno zawiera jeszcze ogon **poprzedniego** akordu, co skutkowałoby zatrzaśnięciem nieprawidłowej nazwy.

Detekcja ataku porównuje poziom sygnału z wolnozmienną obwiednią (EMA), a nie z progiem bezwzględnym, który zależałby od głośności wykonania. Zastosowano ponadto refrakcję 0,2 s, aby pojedyncze szarpnięcie wyzwało dokładnie jeden atak.

8.3. Pomiar czasu zamiast wartości założonej

Licznik postępu otrzymywał wartość stałą $dt = 0,040$, podczas gdy wątek inferencji wymagał 55–90 ms na cykl (inferencja wraz z 40 ms uśpienia). Licznik pracował zatem wolniej od zegara rzeczywistego: próg 0,6 s osiągnąć był po około sekundzie, przy czym wartość zależała od obciążenia maszyny.

Po korekcie, obejmującej pomiar czasu rzeczywistego, wyznaczanie okresu przez wątek inferencji od początku cyklu, zawężenie okna głosowania z 5 do 3 oraz obniżenie domyślnego progu do 0,25 s, przejście trwa **około 0,3 s** zamiast 1,2 s.

8.4. Rzadka reprezentacja jądra CQT

Pełne jądro obejmuje $4097 \times 144 = 589\,968$ wag, skoncentrowanych wokół częstotliwości środkowej każdego binu. Odrzucenie wag poniżej $1e-4$ wartości szczytowej daje następujące wyniki:

próg	zachowanych wag	maks. błąd względem szczytu
------	-----------------	-----------------------------

1e-5	21,9%	0,006%
------	-------	--------

1e-4	6,9%	0,033%
------	------	--------

1e-3 2,3% 0,352%

Błąd wyznaczono na trzech widmach: białym, różowym oraz harmonicznej serii gitarowej. Po transformacie CQT następuje log-normalizacja do zakresu 80 dB, wobec czego wartość 0,03% pozostaje o rzędy wielkości poniżej rozdzielczości cechy.

Rozmiar pliku wag zmniejsza się z **28 MB do 2 MB**, a ścieżka audio wykonuje około **14-krotnie mniej mnożeń** na ramkę. Poprzednia implementacja przechodziła wszystkie 4097 binów FFT dla każdego ze 144 binów CQT, odsiewając wartości zerowe dopiero wewnątrz pętli.

8.5. Zgodność cech pomiędzy trenerem a aplikacją

Rozbieżności tej klasy są szczególnie trudne w diagnozie, ponieważ aplikacja pozostaje funkcjonalna, wykazując jedynie błędy klasyfikacji. Zidentyfikowano dwie:

- **mapowanie chromy.** Plik dystrybuowany z aplikacją związał biny parami $(0,1)$, $(2,3)$, ..., natomiast `librosa.cq_to_chroma` stosuje podział $(1,2)$, $(3,4)$, Co drugi bin trafiał do klasy sąsiedniej, co odpowiada rozmyciu chromy o pół tonu na połowie pasma.
- **klucz pamięci podręcznej.** Nazwa pliku `cache` pochodziła z wyrażenia `abs(hash(ścieżka))`. Interpreter Pythona losuje ziarno funkcji skrótu dla łańcuchów przy każdym uruchomieniu procesu, wobec czego pamięć podręczna nie była wykorzystywana pomiędzy sesjami. Obecnie stosowany jest skrót SHA-1 z nazwy pliku.

Aplikacja **odrzuca** wagi w poprzednim, gęstym formacie, sygnalizując to komunikatem.

8.6. Interfejs użytkownika

Po przeglądzie liczba regulatorów została zredukowana z pięciu do czterech:

regulator	uwagi
Bramka szumu	w dBFS, z miernikiem poziomu w tej samej skali i znacznikiem progu
Pewność akordu	próg dla nazwy akordu (tryb Akordy)
Próg dźwięku	próg dla pojedynczego dźwięku (tryby dźwiękowe)

Czas przytrzymania wymagany czas utrzymania poprawnego akordu

Usunięto regulatory `Tail` (ustawiany z interfejsu i nieodczytywany w żadnym miejscu kodu) oraz `In gain` (którego wpływ znosiła normalizacja wykonywana w obrębie ramki, wobec czego przesuwiał on wyłącznie tę samą nierówność co bramka szumu).

Regulator `Confidence` sterował dwiema różnymi wielkościami jednocześnie, a w trybach dźwiękowych podlegał ograniczeniu dolnemu `.max(0.5)`, w wyniku czego cały zakres 0,1–0,5 dawał identyczne zachowanie. Funkcje rozdzielono.

Bramka szumu operowała uprzednio w liniowej skali RMS 0–0,1, która **nie obejmowała poziomu szumu mikrofonu laptopowego** (RMS 0,05–0,15 po wzmacnieniu). Skala decybelowa $-72\ldots 0$ dBFS zapewnia rozdzielczość w wymaganym zakresie oraz zasięg do pełnej skali.

8.7. Tryb diagnostyczny

```
SOLITITO_DEBUG=1 ./solitito
```

Tryb wypisuje przy każdej predykcji trzy najsilniejsze jakości oraz wektor pitch przeliczony na **interwały względem rozpoznanej prymy**:

```
G m7 | min7=97% sus=0% maj=0% | R96# b25 28 b382# 37 44 b56 594# b616 69 b797# 74
```

Narzędzie rozróżnia przypadek, w którym model nie wykrywa septymy, od przypadku, w którym ją wykrywa, lecz pomija w klasyfikacji. Oba objawy są nierozróżnialne na poziomie nazwy akordu i prowadzą do przeciwnych działań korygujących. Przypadek `Gm7` rozstrzygnięto przy jego użyciu bez ponownego treningu.

9. Hipotezy zweryfikowane negatywnie

Rozdział dokumentuje przypadki, w których pomiar obalił wcześniej przyjęte założenie.

hipoteza	wynik pomiaru
----------	---------------

Rozbieżność normalizacji (w ramce wobec różnica poniżej 1 pp; warstwa `InstanceNorm2d`

globalnej) stanowi główne ograniczenie ją kompensuje

Bariera harmoniczna — trzecia harmoniczna
b3 przypada na b7, wobec czego min7 i min pomiar wykonano na segmentach z etykietami
są nierozróżnialne losowymi

Model nie odwzorowuje danych kontrola TRAIN: 83,7% wobec 63,1% na
treningowych (niedouczenie) walidacji, a więc przeuczenie

Dopasowanie szablonów przewyższy głowicę B = 61,0%
quality (B $\approx$ 75% wobec A = 63%)

Sufit prymy na poziomie 64% wynika z artefakt nagrań solowych; w
voicingów jazzowych bez prymy akompaniamencie 97%

Maskowanie prymy odblokuje jakość
powyżej 73% jakość pozostała na poziomie 72%

Chroma w dystrybuowanym pliku jest `cq_to_chroma` przy 24 binach na oktawę
jednoelementowa, a więc nieprawidłowa również przypisuje jedną wagę na bin;
rozbieżność dotyczyła przesunięcia

Bez zmiennej `ORT_DYLIB_PATH` binarka `RUNPATH=$ORIGIN` z pliku `.cargo/config.toml`
wykorzysta bibliotekę systemową rozwiązywał to zagadnienie

Zależność jest jednoznaczna: **wyniki pomiarów potwierdzały się konsekwentnie, natomiast przewidywania formułowane przed pomiarem okazywały się błędne w sposób systematyczny.** Uzasadnia to przyjętą metodykę opartą na sondach.

10. Decyzje projektowe

10.1. Rozwiązania przyjęte

Generator wypisuje etykiety wprost. Żaden krok przetwarzania nie odtwarza informacji z sygnału.

Weryfikacja etykiet przez niezależny skrypt. Rozwiązanie nadmiarowe względem generatora, zastosowane celowo.

Podział zbioru po źródle. Obniża raportowane wskaźniki o kilkanaście punktów procentowych i jest uzasadniony.

Trzy głowice o rozdzielonych rolach. Tryby dźwiękowe opierają się na wektorze pitch, nie na nazwie akordu.

Rzadka reprezentacja jądra CQT. Korzyść dwojaka: rozmiar pliku wag oraz czas przetwarzania w wątku audio.

Odrzucanie niezgodnych wag przez aplikację. Cicha akceptacja skutkowałaby programem funkcjonalnym, lecz błędnie klasyfikującym.

Napisy wkompileowane, bez biblioteki gettext. Przy kilkudziesięciu napisach zależność systemowa oraz katalogi `.mo` generują koszt przewyższający korzyść.

10.2. Rozwiązania odrzucone

Wyprowadzanie jakości z wektora pitch — zmierzone jako gorsze o 21 punktów procentowych od głowicy quality.

Agregacja czasowa jako sposób poprawy jakości — na kontrolowanej populacji daje około +1 pp. Błędy modelu są skorelowane w czasie: model nie wykazuje wahania pomiędzy oknami, lecz konsekwentnie i z wysoką pewnością wskazuje tę samą nieprawidłową odpowiedź.

Tryb taktowy (zapis przesuwający się w tempie, ocena zamiast bramki) — zaimplementowany, a następnie wycofany. Przyjęto założenie, że trener ma reagować dynamicznie.

Faza 3 treningu — zmierzona jako nieprzynosząca poprawy w trzech przebiegach.

10.3. Zagadnienia otwarte

- **Zbiór testowy z instrumentu docelowego.** Wszystkie wskaźniki dotyczą sześciu wykonawców zewnętrznych oraz dwóch renderów zbioru syntetycznego. Brak jest pomiaru na docelowym torze sygnału.
- **Zmiana `CTX_FRAMES` z 48 na 32** — jedyna modyfikacja architektury o realnym uzasadnieniu (opóźnienie), do zweryfikowania w jednym przebiegu.
- **Zwiększenie ilości materiału z rzeczywistego instrumentu** — jedyny czynnik zdolny zmniejszyć różnicę 6,5 punktu procentowego.

11. Podsumowanie

W wyniku przeprowadzonych prac uzyskano model osiągający 92,4% trafień dokładnych na walidacji wyznaczonej z podziałem po źródle, wydany jako pakiety dystrybucyjne dla dwóch platform.

Zasadniczy przyrost dokładności nie wynikał ze zmian architektury, lecz z czterech ustaleń dotyczących danych:

1. połowa zbioru GuitarSet stanowi improwizację opisaną akordami akompaniamentu,
2. adnotacja `instructed` nie zawiera septym i błędnie klasyfikuje pięćset segmentów,
3. podział zbioru na poziomie segmentów wprowadza przeciek,
4. cele pitch należy wyznaczać z rzeczywistego wykonania, nie z zapisu.

Wymienione cztery zmiany przesunęły wskaźnik `Exact` z 44,8% na 92,4%. Żadna z nich nie dotyczyła struktury sieci.

Dokument opisuje stan na sierpień 2026, wersja 0.2.0. Repozytorium:
<https://github.com/greblus/solitito>